Here is the comprehensive project description for **"CampusDrive AI: The Intelligent Placement Ecosystem."**

This document is structured to serve as a **Project Report**, **Resume Entry**, or **GitHub README**. It highlights both the *Administrative* (Management) and *Generative AI* (Preparation) capabilities.

Project Title: CampusDrive AI

Tagline: An End-to-End Placement Management & AI-Readiness Ecosystem.

1. Executive Summary

CampusDrive AI is a full-stack web application designed to bridge the gap between **campus recruitment management** and **student employability**. While traditional platforms (like Superset) focus solely on logistics (scheduling and tracking), CampusDrive AI integrates **Generative AI** to actively prepare students for the jobs they apply to.

It features a unified dashboard where students can apply for drives *and* immediately access AI-driven tools—such as a **Voice-based Mock Interviewer** and an **ATS Resume Auditor**—to ensure they are job-ready before the actual interview.

---

2. Technology Stack

- **Frontend:** React.js, Tailwind CSS, Puter.js (for Edge AI).
- **Backend:** Node.js, Express.js.
- **Database:** MongoDB (for user profiles, job listings, and interview history).
- **AI Engine:** Google Gemini 1.5 Pro / Flash API (via Node.js) & Puter.js (Client-side).
- **Tools:** Redux Toolkit (State Management), Chart.js (Analytics), Puppeteer (Report Generation).

---

3. Detailed Feature List

Module A: The AI Preparation Engine (The USP)

These features differentiate your project from standard clones.

- **1. AI Voice-to-Voice Mock Interviewer**
    - **Function:** Simulates a real telephonic/video interview.
    - **Tech:** Uses the browser's `Web Speech API` to convert student speech to text, sends it to the Gemini API, and plays back the AI's response using Text-to-Speech.
    - **Details:**
        - Context-aware: Acts as a recruiter from specific companies (e.g., "TCS Technical Lead").
        - Drill-down Logic: If a student gives a vague answer, the AI asks "Why?" instead of moving to the next question.
- **2. Intelligent ATS Resume Scorer (Powered by Puter.js)**
    - **Function:** Audits a student's resume against a specific Job Description (JD).
    - **Tech:** Parses PDF text on the client side and uses Puter.js to analyze keyword density.
    - **Details:**
        - Generates a **Match Score (0-100%)**.
        - Identifies **"Missing Power Keywords"** (e.g., "You missed 'Microservices' which is in the JD").
        - Provides actionable bullet-point suggestions to fix the resume.
- **3. Predictive Coding Sandbox**
    - **Function:** An embedded code editor (Monaco Editor) for DSA practice.
    - **Tech:** Integrates with Judge0 API for code execution.
    - **Details:**
        - Includes an **"AI Hint"** button. Instead of giving the answer, the AI provides pseudocode logic to unblock the student.

Module B: The Management Core (The Foundation)

These features demonstrate your ability to build complex business logic.

- **4. Unified Student Dashboard**
    - **Real-time Job Board:** View and filter drives by Package (LPA), Role, and Eligibility.
    - **Application Tracker:** A Kanban-style view (Applied $\rightarrow$ Shortlisted $\rightarrow$ Interview $\rightarrow$ Offer).
    - **Readiness Graph:** A Chart.js visualization showing the student's mock interview scores over time.
- **5. Admin/TPO Portal**
    - **Drive Management:** Post new jobs with specific criteria (e.g., "Min CGPA 7.0").
    - **Student Analytics:** View which students are "High Risk" (low mock scores) and need intervention.

---

4. How the Project Works (Workflow)

Step 1: Onboarding & Resume Parsing

- The student logs in and uploads their resume.
- The backend extracts the text and stores it in the User Profile in MongoDB.

Step 2: Job Discovery & ATS Check

- The student selects a job (e.g., "Java Developer at JPMC").
- Before applying, they click **"Check Eligibility."**
- The system runs the **ATS Scorer**: if the score is <60%, it warns the student to update their resume.

Step 3: The AI Simulation

- The student clicks **"Start Mock Interview."**
- The AI initializes a session using the *specific* tech stack of the job (e.g., Java + SQL).
- The student speaks their answers. The AI evaluates them in real-time on **Accuracy, Tone, and Confidence**.

Step 4: Feedback & Application

- After the mock session, a **"Performance Report"** is generated.
- If the student is satisfied, they click **"Apply for Drive,"** and their profile is sent to the Admin portal.

---

5. Project Impact (Why it matters)

- **Solved the "Feedback Loop" Problem:** Students usually fail interviews without knowing *why*. This system tells them exactly why (e.g., "You mumbled during the technical explanation").
- **Reduced Recruiter Workload:** By using the ATS filter, only relevant and high-quality resumes reach the TPO/Company.
- **Localized Context:** Can be customized with question banks from specific Mumbai colleges (e.g., VJTI, Atharva), making it highly relevant for local placements.

---

6. How to Present This in an Interview

> "I built **CampusDrive AI** because I noticed that existing platforms like Superset are great for *logistics* but offer zero support for *preparation*. I used the **MERN stack** to handle the complex data management of job applications, and I integrated **Google's Gemini API** to build a 'Virtual Interviewer' that has conducted over 500+ mock interviews.The most challenging part was optimizing the **Voice-to-AI latency**, which I solved by using Puter.js for client-side processing, reducing server load by 40%."

Here is the detailed backend implementation guide for **CampusDrive AI**. This backend acts as the central nervous system, managing data flow between your MongoDB database, the React frontend, and the Google Gemini AI.

1. Backend Architecture (MVC Pattern)

We will use the **Model-View-Controller (MVC)** pattern to keep the code clean and scalable.

- **Models:** Define the structure of your data (Database Schemas).
- **Controllers:** Contain the business logic (e.g., "Ask AI a question", "Apply for a job").

- **Routes:** Define the API endpoints (URLs) that the frontend calls.
- **Middleware:** Functions that run before the controller (e.g., "Is this user logged in?", "Is this a PDF file?").

Recommended Folder Structure:

Plaintext

```
server/
├── config/db.js        # MongoDB Connection
├── models/             # Database Schemas
│   ├── User.js
│   ├── Job.js
│   └── Interview.js
├── routes/             # API Endpoints
│   ├── authRoutes.js
│   ├── aiRoutes.js
│   └── jobRoutes.js
├── controllers/        # Logic
│   ├── authController.js
│   ├── aiController.js
│   └── jobController.js
├── middleware/         # Security & Uploads
│   ├── authMiddleware.js
│   └── uploadMiddleware.js
├── utils/              # Helper functions (e.g., PDF parser)
└── server.js           # Entry Point
```

---

2. Complete List of Routes

Here is every single endpoint your project needs to function fully.

A. Authentication Routes (`/api/auth`)

Handles user registration and secure login.

Method	Route	Protected?	Description
POST	/register	No	Registers a new Student or Admin.
POST	/login	No	Authenticates user and returns a **JWT Token**.
GET	/me	**Yes**	Returns the current user's profile data (using the token).

B. Job Management Routes (`/api/jobs`)

Handles the "Superset" features: posting and applying for jobs.

Method	Route	Protected?	Description
GET	/	**Yes**	Get all active job drives (Student view).
GET	/:id	**Yes**	Get details of a specific job drive.
POST	/	**Admin**	Post a new Job Drive (Admin only).
PUT	/:id	**Admin**	Update job details.
DELETE	/:id	**Admin**	Remove a job posting.

POST	/:id/apply	**Yes**	Student applies for the job. Updates applicants array in DB.
GET	/:id/applicants	**Admin**	View all students who applied for a specific job.

C. AI & Preparation Routes (/api/ai)

Handles the "Preparation" features: Mock Interviews and Resume Scoring.

Method	Route	Protected?	Description
POST	/analyze-resume	**Yes**	Upload PDF $\rightarrow$ Parse Text $\rightarrow$ AI Scores it vs. Job Description.
POST	/interview/start	**Yes**	Starts a new session. AI generates the first question based on role.
POST	/interview/chat	**Yes**	Sends user's answer (text) to AI; returns AI's feedback + next question.
GET	/history	**Yes**	Fetches past interview scores for the student's progress graph.

D. User Profile Routes (/api/users)

Handles profile updates and history.

Method	Route	Protected?	Description
PUT	/profile	**Yes**	Update skills, GitHub links, or portfolio URL.
GET	/applications	**Yes**	List all jobs the student has applied to + their status.

---

3. Implementation Details (The "How-To")

Step 1: The AI Controller (`aiController.js`)

This is the most critical part. It connects your backend to Google's Gemini API.

Prerequisites:

```bash
npm install @google/generative-ai pdf-parse multer
```

Code Logic:

```javascript
const { GoogleGenerativeAI } = require("@google/generative-ai");
const fs = require('fs');
const pdf = require('pdf-parse');

// Initialize Gemini
const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);
const model = genAI.getGenerativeModel({ model: "gemini-1.5-flash" });
// 1. Resume ATS Analyzer
exports.analyzeResume = async (req, res) => {
  try {
    // Read the uploaded PDF file
    const dataBuffer = fs.readFileSync(req.file.path);
    const data = await pdf(dataBuffer); // Extract text

    const prompt = `Act as an ATS (Applicant Tracking System).
```

```
        Analyze the following resume text against the role of a Software Engineer.
        Resume Text: ${data.text}

        Return a JSON response with:
        { "score": Number, "missingKeywords": [Array], "feedback": "String" }`;

        const result = await model.generateContent(prompt);
        const response = await result.response;

        // Cleanup file
        fs.unlinkSync(req.file.path);

        res.json(JSON.parse(response.text()));
    } catch (error) {
        res.status(500).json({ error: "AI Analysis Failed" });
    }
};

// 2. Mock Interview Chat
exports.chatWithAI = async (req, res) => {
    const { history, userMessage } = req.body;

    // Start chat with history context
    const chat = model.startChat({
        history: history, // Array of { role: "user" | "model", parts: "text" }
    });
```

```
    const result = await chat.sendMessage(userMessage);
    const response = await result.response;

    res.json({ reply: response.text() });
};
```

Step 2: Database Models (`models/Interview.js`)

You need to store the interview results to show the "Progress Graph" later.

JavaScript

```
const mongoose = require('mongoose');

const interviewSchema = new mongoose.Schema({
    user: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    role: { type: String, required: true }, // e.g., "React Developer"
    date: { type: Date, default: Date.now },
    score: { type: Number }, // Out of 10
    transcript: [
        {
            question: String,
            userAnswer: String,
            aiFeedback: String
        }
    ]
});
```

```
module.exports = mongoose.model('Interview', interviewSchema);
```

Step 3: Security Middleware (`middleware/authMiddleware.js`)

Ensures that only logged-in students can access the AI features.

JavaScript

```
const jwt = require('jsonwebtoken');

module.exports = function(req, res, next) {
  const token = req.header('x-auth-token');
  if (!token) return res.status(401).json({ msg: 'No token, authorization denied' });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded.user;
    next();
  } catch (err) {
    res.status(401).json({ msg: 'Token is not valid' });
  }
};
```

4. System Data Flow Diagram

1. **Student** speaks into the microphone (Frontend).
2. **React** converts Audio $\rightarrow$ Text.
3. **Axios** sends Text + JWT Token to `POST /api/ai/interview/chat`.
4. **Express** validates the Token.

5. **AI Controller** sends the Text + Chat History to **Gemini API**.
6. **Gemini** responds with a follow-up question.
7. **Express** sends the text back to React.
8. **React** uses Text-to-Speech to "speak" the AI's question.

Summary of Work

To implement this backend:

1. Set up **Express Server** and connect to **MongoDB Atlas**.
2. Create the **User** and **Job** models first.
3. Implement **Auth Routes** (Register/Login).
4. Implement **Job Routes** (CRUD).
5. Finally, implement the **AI Controller** with the Gemini SDK.